

02

Electric Vehicle
Codec Technologies India
Internship 2026

EV Charging Station Locator App

Expo / React Native + Google Maps, k-nearest ranking, Dijkstra charging-stop planner

AT A GLANCE

55 / 55

Jest tests (recorded run)

0 errors

tsc --noEmit, strict

40

Sample stations (synthetic)

1 h 44 min

Modinagar -> Pari Chowk, 20 % SOC

19 min

Dijkstra vs greedy saving

1.4 - 4.3 ms

Route compute time

24.67 km

Connaught Place -> Ghaziabad

42 / 0

Web-demo tile relay ok/fail

VERDICT

TESTS 55/55

RECORDED RUN

DECLARED SYNTHETIC

NOT RUN ON DEVICE

TYPECHECK OK

Eswar Mahalingam
Electric Vehicle Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
github.com/eswarmahalingam

1. Objective and brief mapping

Brief: an app that helps users locate the nearest EV charging stations, built with React Native or Flutter, integrating the Google Maps API, with real-time availability and route optimisation. I built an **Expo / React Native (TypeScript)** app with a strict split between the phone UI (react-native-maps with PROVIDER_GOOGLE, expo-location, Directions API client) and pure-TypeScript decision logic (haversine, k-nearest with filters, station scoring, charging-stop route optimiser, Google polyline codec). The same logic modules are transpiled into a single-file Leaflet web demo so the algorithms can be seen running without a phone.

Data notice (from the README): the 40 stations in data/stations_sample.json are declared-synthetic sample data. Their coordinates are approximate centres of real Delhi-NCR / Ghaziabad localities, but the chargers, operators, tariffs and availability numbers were generated by scripts/make_sample_stations.py (seed 2026). Open Charge Map returned HTTP 403 without an API key and the Overpass mirrors were unreachable from the build environment, so the app ships with the synthetic file and switches to live Open Charge Map data as soon as a key is configured.

2. Architecture

MapScreen -> FilterBar (connector / power / free-now chips), StationMarker (colour = availability), RouteCard (stops, SOC, time breakdown); StationDetailScreen with score-breakdown bars and a Google Maps deep link. Services: stations.ts (Open Charge Map with bundled fallback), availability.ts (Firestore RTDB push / ThingSpeak poll / simulated), directions.ts (road polyline + leg distances). Logic: haversine.ts, nearest.ts, scoring.ts, routeOptimizer.ts, polyline.ts - no React Native imports, so they run under Jest and in the browser demo.

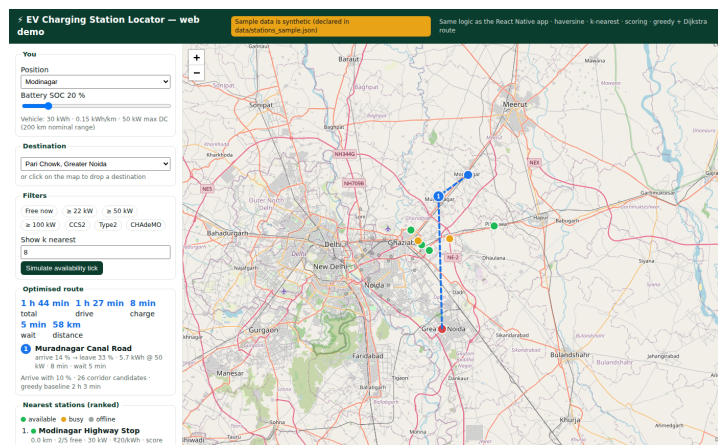


Fig. 1 - Web demo (Leaflet/OSM, headless Chromium 1400 x 860): user at Modinagar with 20 % SOC, destination Pari Chowk, the optimiser's single stop numbered, nearest stations coloured by availability.

3. Method and implementation

- Ranking (scoring.ts): four sub-scores in $[0, 1]$ - distance $1 - d/25$ km, availability (offline 0, unknown 0.3, free $0.5 + 0.5 \text{ free/total}$, busy $0.4 (1 - \text{wait}/60 \text{ min})$), power (fastest outlet / 150 kW), price (linear INR $8 \rightarrow 25$ per kWh) - weighted $0.4 / 0.3 / 0.2 / 0.1$.
- Route planner (routeOptimizer.ts): vehicle = batteryKwh, kWh/km, SOC, maxChargeKw; road distance = haversine $\times 1.3$ until Directions data arrives; reserve 10 %, charge to 80 % at every stop. Step 1 ellipse pruning $d(O,s) + d(s,D) \leq 1.6 d(O,D)$ and ≥ 20 kW online stations only; step 2 greedy baseline; step 3 dense-array Dijkstra on origin + m candidates + destination with edge weight drive + 5 min overhead + wait + charge (energy of the leg / $(\min(P_{\text{station}}, P_{\text{car}}) \times 0.9)) - O(m^2)$; step 4 replay legs and trim the final charge.
- Real-time availability: Firestore RTDB node /availability/{stationId} written by the CPO controller (onValue subscription), or a ThingSpeak channel polled every 30 s with latest-row-per-station parsing, or a simulated random walk every 8 s with an on-screen banner saying so.

4. Results (results/route_examples.json, 30 kWh / 0.15 kWh/km / 50 kW car)

Ranked nearest stations to Ghaziabad centre (results/route_examples.json, nearestToGhaziabad) - the ranking is not pure distance, which is the point of the weighted score:

Scenario	Stops	Total time	Greedy	Distance	Compute
Ghaziabad -> Connaught Place, 80 % SOC	none (arrive 65 %)	45.4 min	45.4 min	30.2 km	4.26 ms, 28 candidates
Modinagar -> Pari Chowk, 20 % SOC	Muradnagar Canal Road: arrive 14 %, +5.7 kWh, 7.6 min charge + 5 min wait	104.4 min	123.3 min	57.9 km (direct 53.4)	2.38 ms, 26 candidates
Modinagar -> Pari Chowk, 12 % SOC	Modinagar Highway Stop: +7.4 kWh, 16.5 min	101.5 min	greedy fails	53.4 km	2.17 ms
Hapur bypass -> Loni, 15 % SOC	unreachable: gap-too-large (10 km usable range, nearest ≥ 20 kW charger 19 km by road)	-	-	-	0.98 ms

KEY TAKEAWAY The station-choice and stop-planning logic is unit-tested with real numbers (55/55, strict TypeScript clean); the phone UI compiles but was not run on a handset, and the 40 stations are declared synthetic.

02

Electric Vehicle
Codec Technologies India
Internship 2026

EV Charging Station Locator App

Expo / React Native + Google Maps, k-nearest ranking, Dijkstra charging-stop planner

AT A GLANCE

55 / 55

Jest tests (recorded run)

0 errors

tsc --noEmit, strict

40

Sample stations (synthetic)

1 h 44 min

Modinagar -> Pari Chowk, 20 % SOC

19 min

Dijkstra vs greedy saving

1.4 - 4.3 ms

Route compute time

24.67 km

Connaught Place -> Ghaziabad

42 / 0

Web-demo tile relay ok/fail

VERDICT

TESTS 55/55

RECORDED RUN

DECLARED SYNTHETIC

NOT RUN ON DEVICE

TYPECHECK OK

Eswar Mahalingam
Electric Vehicle Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
github.com/eswarmahalingam

Scenario	Stops	Total time	Greedy	Distance	Compute
Loni -> Pari Chowk, 25 % SOC, >= 50 kW only	Mohan Nagar Junction: +3.1 kWh, 4.1 min	84.6 min	107.8 min	50.3 km	1.39 ms, 15 candidates

Haversine Connaught Place -> Ghaziabad = 24.67 km. The headless web-demo run relayed 42 CDN/tile requests with 0 failures and reported the same plan as the Jest scenario (1 stop, 104.35 min total, 7.57 min charge, 86.78 min drive, 5 min wait, 57.85 km, greedy 123.27 min, 26 candidates; results/web_demo_run.json). The offline-fallback run (results/web_demo_offline_run.json) hit two net::ERR_FAILED resource loads, detected the missing Leaflet script and drew the equirectangular SVG map instead with an identical plan.

Rank	Station (synthetic)	Distance	Status	Score
1	Ghaziabad Railway Station Parking	1.36 km	available	86
2	Nehru Nagar Community Charger	0.47 km	available	80
3	Raj Nagar Extension Hub	4.40 km	available	74
4	Wave City Expressway Stop	2.82 km	available	71
5	Kavi Nagar Market	1.07 km	busy	58

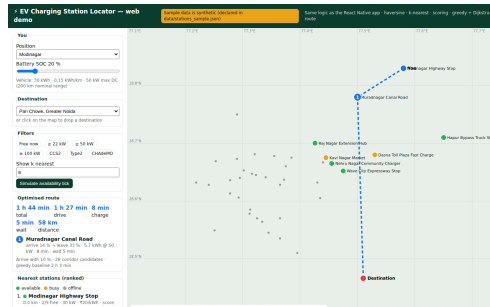


Fig. 2 - Offline fallback render (SVG map, no CDN): same logic, same 40 synthetic stations, same one-stop plan.

5. Verification

Tests were not re-run in this environment: app/node_modules is not installed and npm cannot reach the registry here, so Jest and tsc could not execute. The figures below are the project's recorded run saved in results/test_results.txt (npk jest --verbose, 6 suites, 55 tests, 2.4 s) and results/typecheck_results.txt (npk tsc --noEmit, strict, whole app and logic-only config: 0 errors).

Suite	Tests	What is checked	Recorded
haversine.test.ts	10	zero / symmetry; Connaught Place -> Ghaziabad 24.67 km; London-Paris 343.5 km; 1 deg lat = 111.2 km; antipodes; bearing, cross-track, bounding box	PASS
nearest.test.ts	11	k-nearest ordering, $k > n$ and $k \leq 0$, deterministic ties, every filter alone and combined, every ≥ 50 kW station has CCS2	PASS
scoring.test.ts	7	scores in $[0,1]$; monotone in distance, availability, power, price; weights change ranking; sorted output with breakdown	PASS
routeOptimizer.test.ts	13	no stop when range suffices; legs within range; Dijkstra $\leq$ greedy; prefers fast charger; avoids 45 min queue; never offline; trims last charge; no-candidates / gap-too-large errors	PASS
polyline.test.ts	3	Google documented example decodes exactly; NCR round-trip; empty string	PASS
services.test.ts	11	40 synthetic stations flagged; fallback without key; OCM URL and POI mapping; Indian tariff parsing; ThingSpeak parsing; simulation invariants; Directions URL order; road-distance refinement	PASS

6. Data integrity and declared substitutions

- Not run on a phone or emulator: gestures, map rendering, permission prompts and the Directions round-trip are described from the code, not observed; phone-UI screenshots are deliberately absent (README section 10).
- Sample data is synthetic (seed 2026); real Open Charge Map records replace it automatically with a key. Neither Open Charge Map nor Google exposes live plug status, so availability needs a CPO feed (Firebase / ThingSpeak designs provided).
- Physics are simplified: constant Wh/km, linear charging capped at 80 %, haversine $\times 1.3$ until Directions data arrives; one Directions request per change of route shape.

7. Learnings

Keeping the decision logic free of React Native imports is what made this project verifiable: the same functions run under Jest, in the browser demo and on the phone. The dense $O(m^2)$ Dijkstra is the right choice at 26-28 candidates (1-4 ms); a heap version only wins on sparse graphs. The Modinagar -> Pari Chowk case shows why the greedy baseline is kept: it picks the station closest to the destination and pays 19 extra minutes in charging and waiting. Next: measured charge curves per connector, time-of-day traffic in the drive term, and a device build with a Google Maps key.

KEY TAKEAWAY The station-choice and stop-planning logic is unit-tested with real numbers (55/55, strict TypeScript clean); the phone UI compiles but was not run on a handset, and the 40 stations are declared synthetic.